Predicting Product Sales from Product Features

Project Two's question is more business oriented than last time. To be able to show the ability to work with data sets discussing things that matter in a business setting. Regarding business, what's most useful to them? Huge companies like google, Amazon, especially Uber Eats, take huge amounts of user data to know what customers like and don't like with the goal of understanding what makes products sell better. Services make money through better sales, and being able to predict what would get them better sales is important because it would allow any specific company to focus their resources on the things that matter. Instead of aimlessly selling products and updating apps they could know exactly what leads to higher sales and aims to improve customer satisfaction on that specific thing to raise sales on their end and satisfaction on the customer side. If certain factors like review scores or even category are strongly related to sales, a company could use that information to make better decisions about pricing and advertising. Beyond just creating machine learning models, would the available variables in the dataset contain useful information for predicting sales.

The goal here is to find out whether product information can be used to predict sales. For that purpose, the research question was:

Can product characteristics such as price, category, review score, and review count predict the number of sales, and what factors are associated with increased sales?

The dataset linked below contains information on around **1,000 products**. Each row represents one product, and the columns include product details along with monthly sales data. In hindsight this dataset is relatively small which could've foreshadowed potential inaccuracy. The variables in the dataset I used included category, price, review_score, review_count, sales_month_1 through sales_month_12, annual_score, avg_monthly_score. Of these variables the target variable (the thing I'm trying to predict) is annual_score and the predictor variables are price, category, review_score, and review_count.

Immediately I imported all the necessary libraries required for this project. Matplotlib for making graphs, NumPy for numerical calculations, Pandas for working with the dataset as a table of values, seaborn for cleaning up statistical charts, and Scikit-learn as the machine learning library of choice. First, I loaded the dataset from the zip file storing it in the "df" variable. When observing and cleaning the data to make sure it was properly usable I inspected the structure of the data, the shape, and the data type of each column. For machine learning, checking the data type is incredibly important because by itself machine learning models can't directly understand text categories like category and product_name. This means that later, when building the model I'll have to use a One Hot Incoder (more on that later). However, like the last data set I worked with, (I suppose this is true for a lot of Kaggle datasets), there were no duplicates and the data was already clean. Afterwards I used summary statistics to get another rough outline of the data, checking the mean, median, standard deviation, min and max.

With cleaning the data out of the way, I began to use Exploratory data analysis, the goal being to understand patterns in the data before training the models to make predictions.

```
corr_cols = ["price", "review_score", "review_count", "annual_sales", "avg_monthly_sales"]
corr_matrix = df[corr_cols].corr(numeric_only=True)
corr_matrix
```

[20] ✓ 0.0s

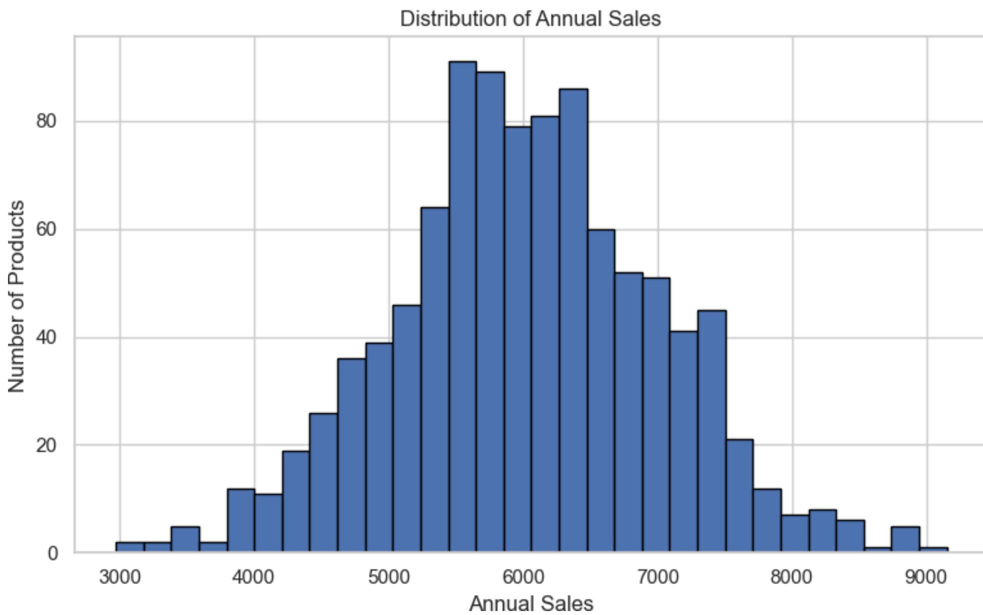
	price	review_score	review_count	annual_sales	avg_monthly_sales
price	1.000000	0.028960	0.042189	-0.015978	-0.015978
review_score	0.028960	1.000000	0.027351	-0.018186	-0.018186
review_count	0.042189	0.027351	1.000000	-0.069393	-0.069393
annual_sales	-0.015978	-0.018186	-0.069393	1.000000	1.000000
avg_monthly_sales	-0.015978	-0.018186	-0.069393	1.000000	1.000000

I began by creating a correlation matrix to measure how strongly two variables move together. A correlation close to one indicates a positive relationship while a correlation close to zero indicates a negative relationship. 0 is the neutral no relationship. A correlation matrix can be great for foreshadowing the results of your machine learning models as they use these correlations for their predictions. Unfortunately, as shown above my correlation results are relatively weak (close to 0). This already hints that my machine learning models may struggle.

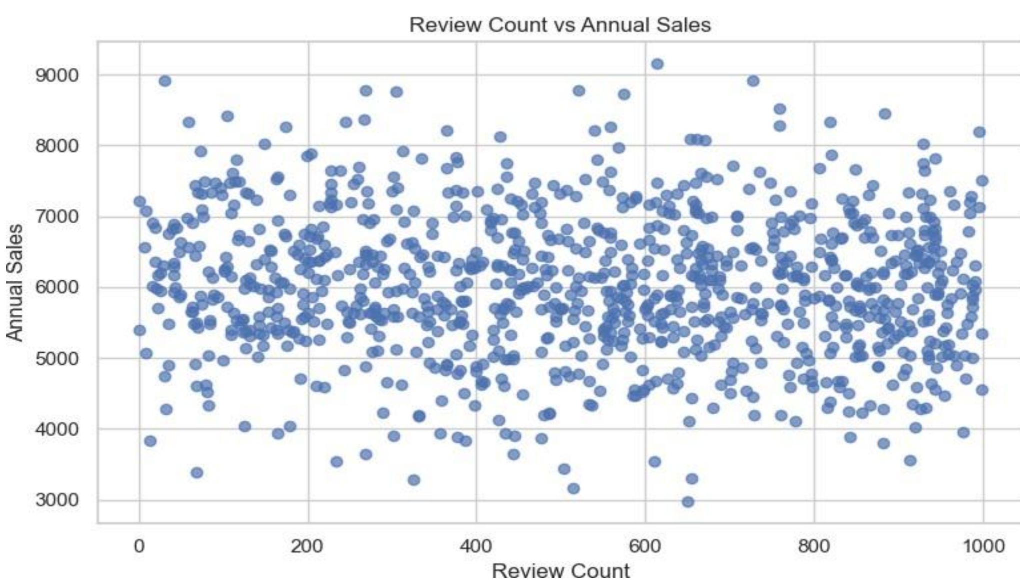
An important side note is the exact positive correlation of 1 for annual sales and avg_monthly_sales. While it seems like a strong indicator it's important to remember that avg_monthly_sales is calculated from annual_sales (annual_sales / 12 = avg_monthly_sales). This means that even though the two are two separate columns, they're basically the same information in different forms. This is why avg_monthly_sales_ could not be used as a predictor of annual_sales as that would be data leakage because the model would be given the answer in a disguised form.

With that out the way my next move was to create a histogram and a dot plot for the purpose of answering these two questions:

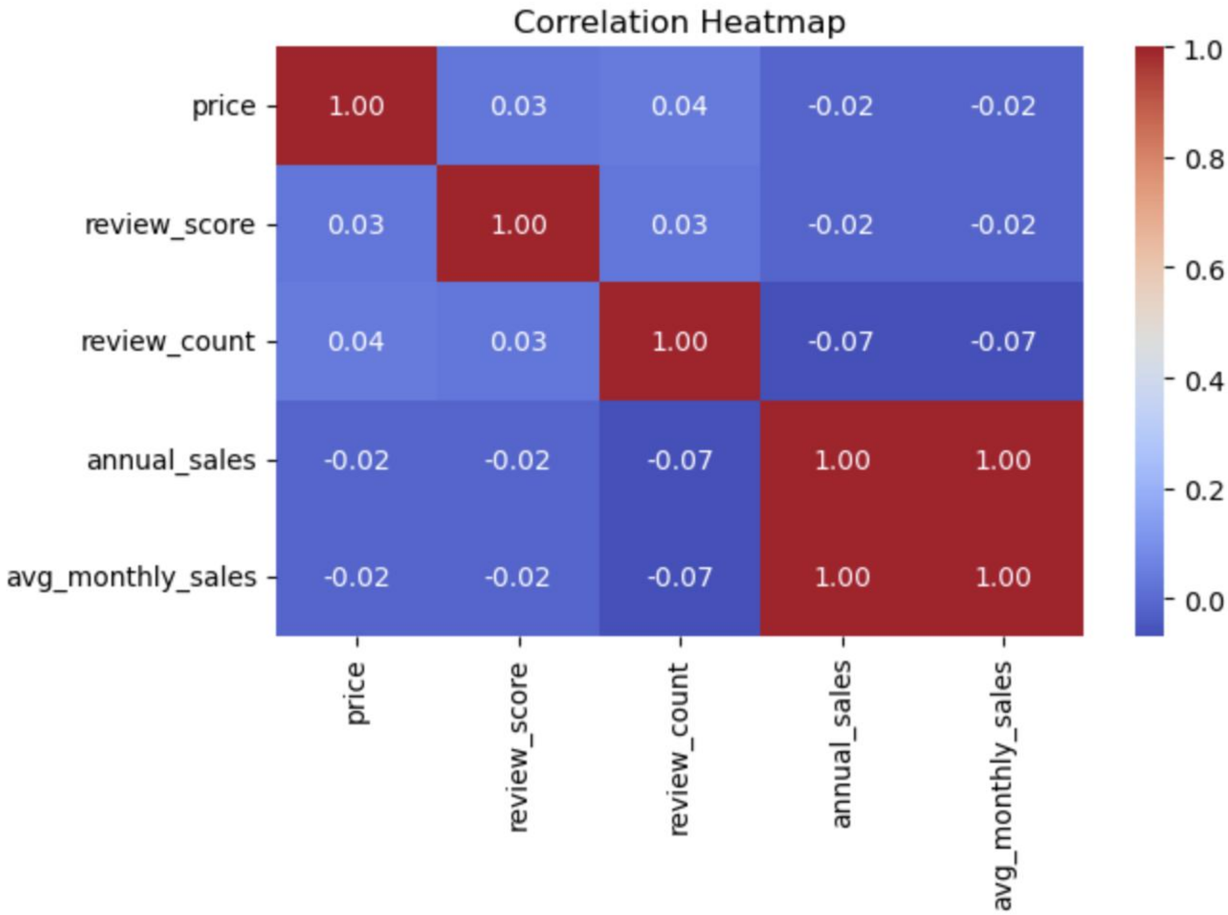
Are most products selling around the same amount, or are there many low-selling and high-selling products?



Do products with more reviews tend to have higher annual sales?



A histogram shows the distribution of one variable, and the scatterplot compares review count against annual sales. While the histogram came out ok and showed a normal distribution indicating that most products were selling around the same amount, the chaotic nature of the scatterplot indicates low to no correlation once again.



And finally a heatmap. A staple because they show strong linear relationships between variables. It's great for visualizing correlations and is basically the visual graph of the correlation chart that we made earlier. Here it's easier to see correlations as positive correlations are warmer (closer to red which is one) and negative correlations are colder (closer to purple which is negative one). However, like last time the correlations between these predictors and annual sales were weak. No strong relationships mean the machine learning models will be less likely to be correct.

With nothing more to check, I proceeded to preprocessing which involves cleaning and transforming the raw data I have so that it can be interpreted correctly. This is where I use a One Hot Incoder telling the computer to treat numerical columns one way and categorical columns another way. One Hot Encoding converts category names into numeric columns so something like “books” becomes “category_books”, represented by a one or zero. With conversion out of the way machine learning can begin.

Machine Learning:

Of the **three regression models** I planned to use I chose **Linear regression** as a baseline model. It is simple, interpretable, and useful for checking whether there is a basic linear relationship between the predictors and annual sales. The goal is to predict annual sales using a linear combination of features. The second was a **Random Forest Regressor.**

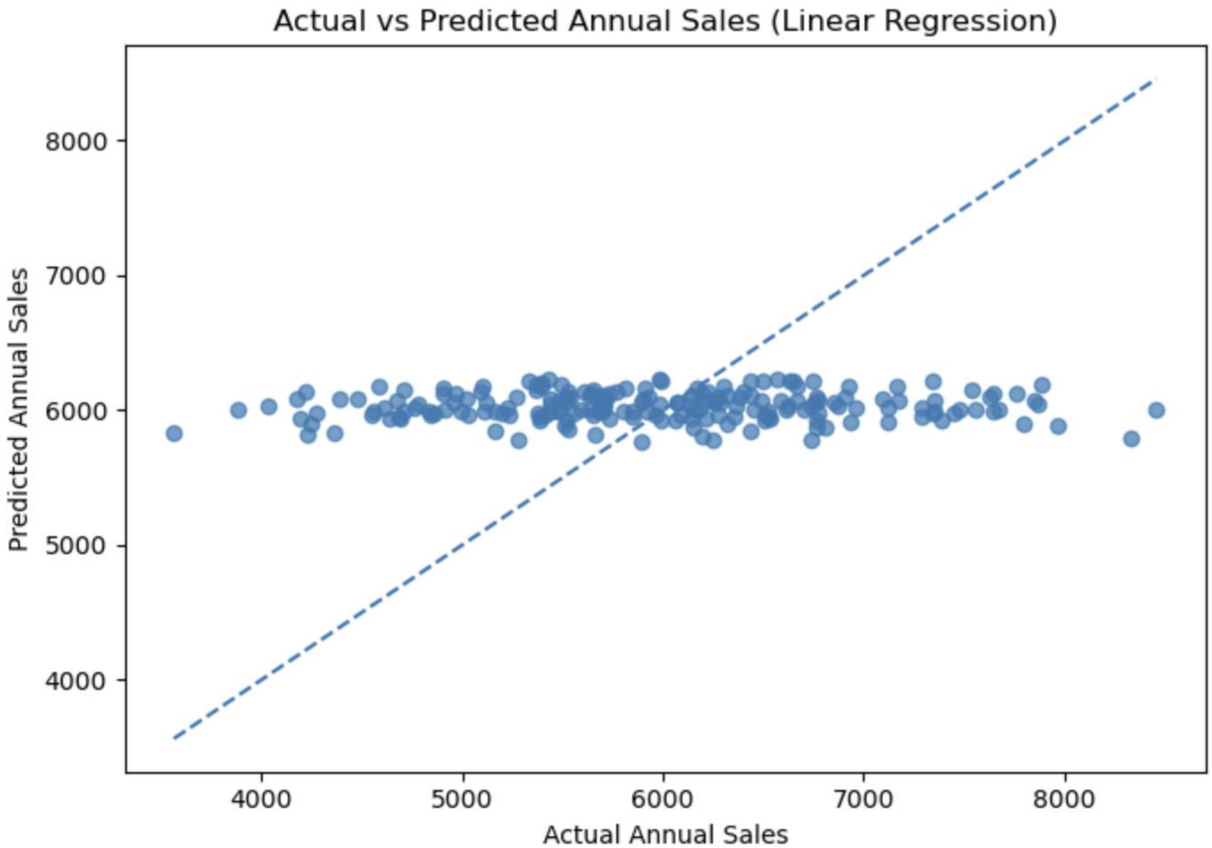
Random forest is a tree-based ensemble model. I chose it because it can capture nonlinear relationships and interactions between variables that linear models might miss. The more trees you use in the forest model the slower but stronger it is. I chose a safe 300 trees and made the “random_state” = 42 which simply makes the results reproducible. The third and final model was **Gradient boosting**, another ensemble model that often performs well on structured datasets. This one is cool because it builds multiple weak decision trees one after another, each new one building off the last one, correcting previous errors. I included it to test whether a more flexible model could detect patterns that the simpler models could not.

When evaluating I split the dataset into training and testing sets. The models were trained on the training set and evaluated on the test set. This is important because it tests how well the model works on unseen data rather than only on the data it learned from. To evaluate performance, I used **MAE (Mean Absolute Error)** which is the measure of the average absolute difference between the true sales and the predicted sales, **RMSE (Root Mean Squared Error):** which is basically prediction error, but it penalizes larger errors more heavily than MAE, and **R^2 (R-squared),** which measured how much of the variation in the target variable is explained by the model. Higher values are better. A value near 1 means strong predictive power, while a value near 0 means weak predictive power. A negative value means the model performs worse than simply predicting the mean.

But data tells the truth. The model comparison results were as foreshadowed by the heatmap and correlation matrix:

- **Linear Regression:**
  MAE = 777.93
  RMSE = 956.90
  R^2 = -0.0068
- **Random Forest Regressor:**
  MAE = 830.72
  RMSE = 1038.51
  R^2 = -0.1858
- **Gradient Boosting Regressor:**
  MAE = 845.32
  RMSE = 1048.37
  R^2 = -0.2084

As stated previously **R^2 (R-squared)** measures how much of the variation in the target variable is explained by the model and is the main thing we look at when determining how well the model does. A value near 1 means strong predictive power, while a value near 0 means weak predictive power. As shown, none of the models even made it past 0. The best model was **Linear Regression**, but its R^2 value was still slightly negative. Unfortunately, even the best model did not explain annual sales well and ultimately this means that **the available features were poor predictors of annual sales**.



I believed that it made sense that products with better reviews and more reviews would have higher sales, but the model's failure suggests that these variables alone do not capture enough information. I made one last scatterplot this time comparing actual sales to predicted sales. On the scatter plot the x axis is actual annual sales and the y axis is predicted annual sales. The diagonal line represents perfect predictions and if the model were accurate the points would cluster along the diagonal line but as can be seen that was not the result. This graph shows the model's predictions were incredibly weak. This could be because Important variables are missing, relationships were weak from the beginning (likely), or the dataset was too small (models had too limited information to learn from). The negative R^2 values are especially important because they show that the models are not just imperfect; they are failing to capture a useful predictive signal from the available inputs. With **ethical considerations** in mind If a model performs poorly as in this case the honest conclusion is that the current variables are not enough for accurate sales prediction, the models are unusable. **Despite that**, a negative result can still be meaningful in data science, explaining that to have better predictions we require more data or better variables rather than just more complex models.

AI Transparency: Artificial Intelligence was used in writing, making sure definitions were accurately stated and ethical guidelines were followed